

Lab 04 - State Variants, Clean-Up and Reframing with Masks

Course: Create Short Video Film using AI (TGS-2020505925) | Version v12.0 | 27 September 2026 Topic 02: AI Image Processing, Generation and Visual Enhancement Outcome: ELO2 | In-class time: 15 minutes of scheduled practical time

Competency	Statement
Ability A2	Apply the principles of processing, filtering and analysis methods for video data
Knowledge K4	Image and video processing, filtering and transformation methods

Scenario

The HARBOUR PIER plate needs three edits before it becomes a reference: remove a stray traffic cone, make a second story state ('the ferry has pulled away - change nothing else'), and deliver a 9:16 version. A model returned the state edit; you verify it with pixel evidence before accepting it.

What you will produce

An inpainted plate with proof that nothing outside the mask moved, a verification heat-map and verdict for the state edit, and a 9:16 crop and outpaint mask with the percentage of pixels each approach keeps or must generate.

Tools: Python 3 | opencv-python | NumPy | optional: an image editor or generative edit tool

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: opencv-python>=4.10, numpy.

API currency. Everything taught here runs on the base opencv-python wheel plus NumPy - no contrib modules and no downloaded model weights. SSIM is implemented in NumPy because cv2.quality is contrib-only. Generative steps are optional: every lab supplies SIMULATED reference media so it can be completed offline, and asks you to record the model ID and date if you generate your own.

Workflow

1. Mask and inpaint the stray cone
2. Prove nothing outside the mask moved
3. Verify the state edit against its permitted zone
4. Build the 9:16 crop and outpaint mask

5. Accept, regenerate or re-edit

Files in this folder

Path	What it is
reference/pier-docked.png	HARBOUR PIER state A with a stray orange cone (SIMULATED)
reference/pier-leaving-edit.png	State B as returned by a model: ferry leaving, plus one unrequested change (SIMULATED stand-in)
data/edit-jobs.json	Mask boxes, permitted edit zone, thresholds and targets
data/state-edit-prompt.md	The exact state-edit prompt that produced the edit
edit_lab.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom illustration of *The Last Ferry*, rendered deterministically with OpenCV and labelled on the frame, or an AI-generated reference copied unchanged with a PROVENANCE.md beside it. Nothing in this folder may be relabelled: a simulated stand-in is never presented as model output or as a photograph.

Steps

1. Read data/edit-jobs.json. The mask convention here is WHITE = edit, BLACK = preserve. Note that some tools use the opposite - always check the contract.
2. Inpaint the cone. The script builds the mask from the box, dilates it by 4 px and runs two classical inpainting methods.

```
python3 edit_lab.py inpaint --image reference/pier-docked.png --jobs data/edit-jobs.json --out out
```

3. Confirm both methods leave 0 target (orange) pixels and a maximum change OUTSIDE the mask of 0. Open out/inpaint_telea.png and judge the planks by eye.
4. Read data/state-edit-prompt.md. The prompt asked for exactly one change: the ferry has left.
5. Verify the returned edit. Only the ferry zone (allowed_box) may change.

```
python3 edit_lab.py verify --before reference/pier-docked.png --after reference/pier-leaving-edit.png --jobs data/edit-jobs.json --out out
```

6. Read the verdict. The script finds an unrequested region at about [711, 48, 120, 120]. Open out/edit-heatmap.png: what changed there?
7. Decide: regenerate the state edit, or fix the lamp locally with a second masked edit? Write the decision and the prompt fix in out/edit-decision.md.
8. Reframe for vertical delivery.

```
python3 edit_lab.py reframe --image reference/pier-docked.png --target 9:16 --out out
```

9. Compare the two options: a centre crop keeps about 30% of the width and generates nothing; an outpaint keeps 100% of the source but must generate about 70% of the pixels (white in out/outpaint_mask.png).

10. Record which option you will use for the pier in the 9:16 cut-down, and why, in out/edit-decision.md.
11. Optional extension: send outpaint_placeholder.png and outpaint_mask.png to a generative-expand tool, save the result and run verify against the placeholder with an allowed box covering only the new areas.

Verify

Inpainting leaves 0 orange pixels and 0 change outside the mask; verify returns REGENERATE and names one unrequested region near the lamp; reframe reports both percentages.

Expected outputs

- out/cone-mask.png, inpaint_telea.png, inpaint_ns.png
- out/edit-heatmap.png and out/edit-verify.json with the verdict
- out/reframe_crop.png, outpaint_mask.png, outpaint_placeholder.png
- out/edit-decision.md

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ Inpainting output showing zero change outside the mask
- ☐ Verification heat-map with the unrequested region marked, and the verdict
- ☐ Crop-versus-outpaint comparison with both percentages
- ☐ Written accept / regenerate / re-edit decision with the revised prompt

Troubleshooting

Symptom	What to do
'before/after sizes differ'	An edit must not resize the plate; re-export at the original size.
verify finds rain streaks as changes	Raise pixel_threshold slightly or rely on min_region_px; rain is re-rendered noise.
Inpainting smears a large area	Classical inpainting diffuses neighbours; use a generative inpaint for large or textured regions.
My tool's mask is inverted	Invert with cv2.bitwise_not(mask) and state the convention in your notes.

References used in this lab

- [S6] Youri van Hofwegen, 'EXACTLY How to Make an AI Short Film (Full Workflow)', YouTube, 1 Jun 2026, youtube.com/watch?v=0v534yAyhwg - *Full video, 16:49*.

Reference-first principle (stop giving the model freedom); three reference types - characters, locations, props; casting attributes (genre, budget, era, archetype, identity, physique, details, outfit); a second, damaged-state location made by editing the first ('change nothing else'); director controls (genre, camera movement, speed ramp, duration, aspect ratio, audio); per-character emotion and intensity; the multi-shot prompt framework with dialogue in quotation marks; video reference versus start/end frame (mood carries over); three generations for dense action shots; assembly in a timeline editor.

- [S13] M. Tschochohei and F. Schenker, 'Multimodal Generative AI in the Enterprise: From Pixels to Profit', Packt, June 2026, ISBN 978-1-80611-167-1 - *Ch.5 pp.56-75; Ch.7 pp.106-119; Ch.8 pp.127-143*.

VAE, GAN and diffusion mechanisms and failure modes; prompt structure and subject lock; binary masks (white = editable), inpainting, outpainting and mask-free editing; spatial versus temporal coherence, 3-D U-Nets and diffusion transformers over spatiotemporal patches; long-running operation polling; responsible-AI risks (voice cloning, deepfakes, likeness) and transparency (C2PA Content Credentials, invisible watermarking, model cards).

- [S15] Google AI for Developers - Gemini API image generation documentation -
<https://ai.google.dev/gemini-api/docs/image-generation> (verified 6 Sep 2026).

Native image models for text-to-image and conversational image editing; supported aspect ratios including 16:9, 9:16, 4:5 and 21:9; output sizes up to 4K by model; every generated image carries a SynthID watermark.

- [S17] OpenCV 4.13 Python package and FFmpeg, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92); ffmpeg/ffprobe optional*.

Every lab command runs on the base opencv-python wheel plus NumPy: VideoCapture / VideoWriter, cvtColor, calcHist/compareHist, Canny, ORB_create + BFMatcher, HuMoments, inpaint, CLAHE, filter2D, fastNlMeansDenoisingColored, Haar cascades from cv2.data.harcascades, HOGDescriptor_getDefaultPeopleDetector, CamShift and calcOpticalFlowFarneback. SSIM is implemented in NumPy because cv2.quality is contrib-only. FFmpeg is used only when present, to mux audio into the final export.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v12.0.